# User info extraction abusing placeholder injection in Zendesk

**User info extraction abusing placeholder injection in Zendesk** - Rikesh Baniya, Medium.


- Published: 2025-01-14
- Original: <https://rikeshbaniya.medium.com/tale-of-zendesk-0-day-and-a-potential-25k-bounty-61bcf9c5dc06>
- Preserved from: https://rikeshbaniya.medium.com/tale-of-zendesk-0-day-and-a-potential-25k-bounty-61bcf9c5dc06 (live) on 2026-08-09
- Licence: unknown




## Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Bug Bounty

Hackerone

Bug Bounty Tips

Bug Bounty Writeup

Security Research

# User info extraction abusing placeholder injection in Zendesk

[[image: Rikesh Baniya]](https://rikeshbaniya.medium.com/?source=post_page---byline--61bcf9c5dc06----------------------------------)

Rikesh Baniya

In this blog, I will share how I found template injection affecting Zendesk customers with default configuration.

One fine day I came across this writeup shared by Rojan Rijal dai, where he demonstrated how it was possible to exfiltrate sensitive data in support portals like Zendesk, salesforce, etc

https://www.ophionsecurity.com/post/abusing-dynamic-placeholders-in-helpdesks-to-extract-user-data

In the blog, he explained how we can use Zendesk placeholders/templates to create a ticket, and when the ticket gets created, the templates would get rendered allowing data exfiltration.

Zendesk has a wide range of templates like: `{{ticket.ccs[0].name}}` `{{ticket.ccs[0].phone}}` `{{ticket.ccs[0].custom_fields}}` `{{ticket.ccs[0].notes}}` which can be used during support ticket creation.

https://support.zendesk.com/placeholders-reference

The impact of the issue was:

An attacker could create a support ticket containing malicious templates with the victim user in CC.

When the template gets rendered, the attacker could exfiltrate the CCed user's phone, name, internal notes, custom fields, etc.

Exfiltrating custom fields and user notes of CCed user could allow an attacker to access internal data, including user addresses, billing information, and even PII.

It was a fascinating attack vector to learn and I wanted to test it against my private programs.

One key point highlighted by Rojan dai was:

- This can only be exploited against custom support portals.

For example ; https://support.gitlab.com/hc/en-us is a default Zendesk portal.

Default Zendesk portal use Zendesk's own form

https://support.gitlab.com/hc/en-us/requests/new to create tickets.

Whereas https://support.github.com is a custom support portal with Zendesk in the backend.

Custom portals are just sites with custom form, the form data is later passed to Zendesk via api, thus injection was possible if the programs were using `/v2/tickets.json` api in the backend to create tickets.

**Starting the hunt**

Trying to test this issue against most of my programs quickly came with a realization that its a challenge to identify these "custom forms" and required extensive manual checking.

On the contrary, identifying the default Zendesk portal was only a matter of minutes.

All I needed to do was gather subdomains of my programs and after CNAME resolution I found tons of default Zendesk portals across all my bug bounty programs.

i.e `target.zendesk.com` subdomain of my target programs.

Since the attack was less explored against default portals in the write-up, I decided to test that route.

**Testing the portals**

This is what a normal zendesk portal (default) looks like with the fields required being:

`email, subject and description`

I submitted the form with malicious templates in the subject/description.

After the ticket was created, although user input was getting reflected, the injection wasn't possible due to sanitization.

Zendesk was **sanitizing the subject content.**

What if I create a ticket with no subject instead? With just the payload in description body.

Aha, the subject can’t be blank!

It felt like reaching a dead end even before I could start.

As I was going through a few more of those default portals i noticed something interesting.

One of the programs didn’t require a subject field in their Zendesk portal to create a ticket.

This time I could just submit the form with a malicious template in the description.

**Suprise Surprise !!!**

The template got rendered.

So although zendesk was sanitizing the **original subject field** , if a ticket is created with no subject , it will use the description body as the subject instead.

This `description <-> subject` referencing lacked sanitization.

We now had a placeholder injection in Zendesk.

But there was a small problem, this could only be exploited against programs that didn't have a subject field in their Zendesk form.

And that number was few.

With the beauty of this bug , i hadddd to bypass this mandatory “**subject**” field requirement and be able to create tickets with blank subject and payload in description.

**Email based ticket creation**

Along with api/form based ticket creation, zendesk also allows email based ticket creation.

We can send an email to `support@target.zendesk.com` , the email content would then be handled as a support ticket.

We can also add other users as CC to the ticket from the email itself.

Since i can control the subject/body field when sending an email, i sent an email to `support@target.zendesk.com` with no email subject , malicious template in email body and victim email as CC.

What do we get as an email reply??

A beautiful template injection allowing me to exfiltrate victim info by adding them as CC ; `name, phone, role, custom fields` and much more

**Reporting**

The issue had a range of impact depending on the program.

Some programs had already disabled the Zendesk email CCing feature for end users, thus no info extraction possible.

Where as some programs were using **static subject line**. With no attacker input being handled, injection wasn't possible at all.

*Static email subject*

Zendesk also has a template suppression/activation rules.

So the programs did have measures they could implement against this attack.

**Program Response**

The issue was handled as low-high impact issue depending on what data i was able to exfiltrate.

Almost all of the programs were very proactive towards the issue and took quick steps to resolve it.

In the mean time few programs asked me to report it to Zendesk instead and also made communication from their end.

The issue was then reported to Zendesk.

It was rewarded as “Medium — Business Logic Issue” / 750$ .

But I still believe it to be atleast a high impact issue.

Regardless, a total of 13,300 $ was earned from 17 reports submitted to individual vulnerable programs.

I still had around 15–20 reports open, which could have easily brought in over $25K if the issues had been handled similarly by individual programs.

But considering the number of programs impacted, it was ultimately better for everyone with Zendesk coming up with the fix.

Kudos to the team for swift resolution and also to the individual programs for bounty and measures taken from their end.

It was a thrilling ride, from research to exploit to reporting.

---